

Ocean Across

Full-Stack Developer Assignment

Overview

Build a **Sessions Marketplace** web application where users can sign in via OAuth, browse sessions, and book them. This assignment tests your end-to-end skills across frontend, backend, database, and deployment.

Tech Stack (Required)

- **Frontend:** React or Next.js (client-side only).
- **Backend:** Django + Django REST Framework.
- **Database:** PostgreSQL.
- **Infrastructure:** Docker (multi-container: frontend, backend, database, reverse proxy).
- **Authentication:** OAuth (Google or GitHub) with JWT issued by backend.

Core Features

Authentication & Roles

- OAuth login via Google or GitHub.
- Backend issues JWT tokens for frontend use.
- Two roles:
 - **User** – browse and book sessions.
 - **Creator** – create and manage sessions.
- Profile page: view/update basic details (name, avatar).

Sessions & Booking

- Public catalog of sessions.
- Session detail page with “Book Now” option.
- Booking flow records user bookings.
- User can view active and past bookings.
- Creator can manage their own sessions.

Dashboards

- **User Dashboard:** bookings and profile.
- **Creator Dashboard:** sessions management and booking overview.

Bonus Features (Optional, Extra Credit)

- **Payment Gateway Integration:** Stripe or Razorpay (test mode).
- **MinIO / S3 Uploads:** For storing images/files.
- **Rate Limiting:** Apply to sensitive endpoints.

Pages to Implement

1. Home / Catalog – list sessions, login CTA.
2. Session Detail – show session info, booking option.
3. Auth Flow – OAuth login, JWT handling.
4. User Dashboard – bookings and profile.
5. Creator Dashboard – manage sessions & view bookings.

Scoring Criteria (100 pts)

Category	Points
Architecture & Docker (clear structure, reproducible compose)	20
Auth & Roles (OAuth + JWT, role enforcement)	20
Core Features (sessions CRUD, booking, dashboards)	30
Frontend UX (clean, responsive, error handling)	15
Code Quality & Documentation (.env.example, README)	15
Bonus: Payment gateway, MinIO, rate limiting	+15

Docker Requirements

- Use **docker-compose** with separate containers for:
 - Frontend (React/Next.js)
 - Backend (Django/DRF)
 - Database (PostgreSQL)
 - Reverse proxy (Nginx)
- One command `docker-compose up --build` should start the system.
- Provide a sample `.env.example` file with all required variables.

Deliverables

- GitHub repository with frontend and backend code.
- Docker setup that runs with one command.
- `.env.example` file with all required environment variables.
- README including:

- Setup steps (cloning, env vars, docker commands).
- OAuth client setup instructions.
 - Example demo flow (login, create session, book session).